Apuntes de preparación a partir de ejercicios para primera prueba evaluable de Java con Marga

- Apuntes de preparación a partir de ejercicios para primera prueba evaluable de Java con Marga
  - **Ejercicio 1 - Tipos de datos - Validar identificadores**
  - **Ejercicio 2 - Literales - Validar literales**
    - **A. Investiga como se representan los números en los distintos sistemas de numeración**
    - **B. Indica cuáles de los siguientes literales son válidos en Java. Si es válido, indica de qué tipo es (int, double, long, etc) y el sistema de numeración en el que está escrito (decimal, binario, octal, decimal). Si el literal no es válido explica por qué**
      - ◇ Tipos básicos de números en Java y puntos clave
      - ◇ Reglas generales para saber si un literal es válido
      - 💡 Conclusiones clave
      - 📋 **EJEMPLO RESUELTO:**
    - **C. Indica cuáles de los siguientes literales de tipo char son válidos en Java. Si el literal no es válido explica el motivo.**
      - 📋 **RESUMEN DE REGLAS PARA LITERALES DE TIPO CHAR EN JAVA**
      - **EJEMPLO RESUELTO**
  - **Ejercicio 3 - Declaraciones de variables**
    - A. Escribe la declaración para cada una de las variables y asígnales un valor inicial en la propia declaración de variable
    - B. Escribe la declaración más apropiada para cada una de las siguientes variables. El nombre de cada una indica el tipo de dato que contendrá y servirá para determinar el tipo de dato más adecuado en la declaración de la variable
  - **Ejercicio 4 - Tipodato en expresiones aritméticas y Conversiones**
    - **A. A partir de las siguientes declaraciones de variables**
      - **Determina cuál es el tipo de dato del resultado de las siguientes expresiones aritméticas**
    - **B. A partir de las siguientes declaraciones de variables**
      - **Indica en cada caso si la instrucción es válida o si se requiere una conversión explícita (casting) para que compile**
        - **Tabla resumen sobre cuando se quiere CASTING**
        - **Tabla resumen sobre Conversión entre tipos de datos en Java**
          - 💡 **Ejemplos prácticos**
  - **Ejercicio 5 - Operadores lógicos**
  - **Ejercicio 6 - Operadores relacionales**
    - **A partir de las siguientes declaraciones de variables**
      - **Determina el valor de las siguientes expresiones**
  - **Ejercicio 7 - Expresiones algorítmicas**
    - **Escribe las expresiones algorítmicas equivalentes para cada uno de los siguientes enunciados**

Ejercicio 1 - Tipos de datos - Validar identificadores

> **Reglas de validación de identificadores**
>
> - Debe comenzar por una letra (A-Z o a-z), un guión bajo "_" o un signo dólar "$".
> - No contener espacios, símbolos de puntuación ni carácteres especiales excepto "_" y "$".
> - No puede ser una clave reservada de java, ej: class, for, static, null...

Indica cuál de los siguientes identificadores son válidos en Java. Si el identificador no es válido, explica por qué.

```
int registro1;          //  Valido
int 1registro;          //  Debe comenzar con una letra, un guión bajo o un signo
de dolar.
int archivo_3;          //  Valido
int while;              //  No válido, es una clave reservada de JAVA.
int $impuesto;          //  Valido.
int año;                //  Valido, aunque no recomendado.
int primer apellido;    //  No válido, no puede contener espacios.
int primer_apellido;    //  Valido.
int primer-apellido;    //  No válido, contiene caracteres que no son "_"
int primerApellido;     //  Válido
// int Tom's;           //  No valido, no puede contener caracteres especiales que
no sean _ y $
int C3PO;               //  Valido.
int 123#;               //  No valido, no puede empezar con numeros ni contener
caracteres especiales que no sean "_" o "$"
int PesoMáximo;         //  Valido.
int %descuento;         //  No válido, debe comenzar por una letra, guión bajo o
signo dolar.
int Weight;             //  Válido.
int $$PrecioMínimo;     //  Valido, puede comenzar por signos dolar "$"
int _$único;            //  Valido.
int tamaño_máximo;      //  Valido pero no recomendable usar ñ y tildes
int peso.maximo;        //  No valido, el punto es un caracter especial que indica
atributo o funcion de un objeto.
int Precio__;           //  Valido
int matrícula?;         //  No valido, contiene caracteres especiales que no son
"_" o "?"
int cuántoVale;         //  Válido, no recomendable usar acentos
int high;               //  Valido
int barça;              //  Válido.
int piragüista;         //  Válido
int B_011;              //  Válido.
int X012AB;             //  Válido.
int 70libro;            //  No válido, debe comenzar por una letra, "_" o "$"
int nombre&apellido;    //  No válido, no puede contener caracteres especiales que
no sean _ o "$" además el "&" es un operador que se utiliza para comparar

int 0X1A;               //  No válido, comienza con números.
int else;               //  No válido, es una clave de java.
```

Ejercicio 2 - Literales - Validar literales

A. Investiga como se representan los números en los distintos sistemas de numeración

Java permite escribir **números literales** en varios sistemas numéricos. Aquí tienes los **cuatro principales**:

Sistema	Prefijo en Java	Ejemplo	Base	Explicación
Decimal	*(sin prefijo)*	`123`	10	El habitual, solo dígitos del 0 al 9
Binario	`0b` o `0B`	`0b1010`	2	Solo usa 0 y 1
Octal	`0` *(cero inicial)*	`0123`	8	Usa dígitos del 0 al 7
Hexadecimal	`0x` o `0X`	`0x1A3F`	16	Usa 0–9 y A–F (o a–f)

B. Indica cuáles de los siguientes literales son válidos en Java. Si es válido, indica de qué tipo es (int, double, long, etc) y el sistema de numeración en el que está escrito (decimal, binario, octal, decimal). Si el literal no es válido explica por qué

◇ Tipos básicos de números en Java y puntos clave

Tipo	Tamaño	Ejemplo	Notas
`int`	32 bits	`123`, `0x1A`	Por defecto para enteros
`long`	64 bits	`123L`, `0b1010L`	Debe acabar con `L` o `l`
`float`	32 bits	`1.23f` o `1.23F`	Debe acabar con `f` o `F`
`double`	64 bits	`1.23`, `1.23e5`, `2E-3`	Por defecto para decimales

◇ Reglas generales para saber si un literal es válido

1. 🔢 **No puede haber dos puntos decimales (`.`)**
2. 💣 **Los guiones bajos (`_`) no pueden ir:**
   - Al principio o final del número
   - Justo antes o después de un punto decimal o una "E/e"
   - Antes o después de un prefijo (`0x`, `0b`, `0`)
3. 🧮 **Los exponentes (`E` o `e`)**
   - Solo se usan con decimales o notación científica
   - Van seguidos de un número entero (con o sin signo)
4. 🤯 **Los literales hexadecimales, binarios y octales**
   - **no** admiten letras más allá de A–F (en hex)
   - **no** admiten punto decimal ni "E/e"

💡 Conclusiones clave

Condición	Tipo de número	Excepción

Condición	Tipo de número	Excepción
LLeva '.' o 'e'	`double`	Que tenga 'F' (`float`)
Termina en `l`	long	No
Lleva `0b`, `0`, `0x`	`Binario`, `Octal` o `Hexadecimal`	No
Los sistemas no decimales no aceptan punto decimal ni exponente	64 bits	`1.23`, `1.23e5`, `2E-3`

EJEMPLO RESUELTO:

```
| Nº | Literal      | ¿Válido? | Tipo     | Sistema     | Explicación
|
| -- | ------------ | -------- | -------- | ----------- | ------------------------
---------------------- |
| 1  | 0.5          | ☑        | double   | Decimal     | Decimal con punto,
tipo double por defecto     |
| 2  | .5           | ☑        | double   | Decimal     | Comienza con punto
decimal, válido             |
| 3  | 9.3e-12      | ☑        | double   | Decimal     | Notación científica
con exponente negativo      |
| 4  | 0.8E+0.8     | ❌       | –        | –           | Exponente debe ser
entero, no decimal          |
| 5  | 12345678     | ☑        | int      | Decimal     | Entero decimal sin
sufijo                      |
| 6  | 12345678_L   | ❌       | –        | –           | Guion bajo antes del
sufijo L no permitido       |
| 7  | 234567L      | ☑        | long     | Decimal     | Entero largo (L)
|
| 8  | 05_15        | ☑        | int      | Octal       | Octal válido (0
inicial y _ entre dígitos)      |
| 9  | 018CDF       | ❌       | –        | –           | Octal solo permite
dígitos 0–7                 |
| 10 | 0x87e3a      | ☑        | int      | Hexadecimal | Hex válido, letras
minúsculas permitidas       |
| 11 | 0X12AG       | ❌       | –        | –           | G no permitido en
hexadecimal                   |
| 12 | 0b111        | ☑        | int      | Binario     | Binario válido
|
| 13 | 0B1212       | ❌       | –        | –           | Binario solo admite 0
o 1                         |
| 14 | 1_234.2E-2   | ☑        | double   | Decimal     | Guion bajo correcto,
exponente válido            |
| 15 | 1234.2EF     | ❌       | –        | –           | Exponente debe ir
seguido de número, no F      |
| 16 | 1234.2E3F    | ☑        | float    | Decimal     | Notación científica
con sufijo F válido          |
| 17 | 3e12         | ☑        | double   | Decimal     | Notación científica
```

```
válida                    |
| 18 | 3_e12       | ❌       | –        | –           | _ no permitido antes
de e                      |
| 19 | _234        | ❌       | –        | –           | No puede comenzar con
_                         |
| 20 | 1_2_3       | ☑       | int      | Decimal     | Guiones bajos válidos
entre dígitos             |
| 21 | -3E-12      | ☑       | double   | Decimal     | Valor negativo con
notación científica       |
| 22 | 1.34.5      | ❌       | –        | –           | Dos puntos decimales →
inválido                  |
```

C. Indica cuáles de los siguientes literales de tipo char son válidos en Java. Si el literal no es válido explica el motivo.

📋 RESUMEN DE REGLAS PARA LITERALES DE TIPO CHAR EN JAVA

Regla	Explicación	Ejemplo válido	Ejemplo inválido
1️⃣ Debe ir entre comillas simples `' '`	Un literal `char` siempre se encierra entre comillas simples	`'a'`, `'Z'`, `'ñ'`	`a`, `"a"`
2️⃣ Solo puede contener **un carácter**	Si hay más de uno, no es un `char`	`'X'`	`'XY'`, `'xyz'`
3️⃣ Se pueden usar **caracteres Unicode**	Se usa `\u` seguido de 4 dígitos hexadecimales	`'\u0066'` → `'f'`	`'\u00G1'` ❌
4️⃣ Se permiten **secuencias de escape válidas**	Para representar tabulación, salto de línea, etc.	`'\n'`, `'\t'`, `'\\'`, `'\''`	`'/n'`, `'\ñ'`
5️⃣ No se permiten **comillas dobles (")**	Las comillas dobles indican un `String`, no un `char`	`'A'`	`"A"`
6️⃣ Se permiten **caracteres especiales y símbolos**	Cualquier símbolo individual es válido	`'?'`, `'+'`, `':'`, `'$'`	`'??'`
7️⃣ Los `char` pueden representar cualquier carácter Unicode	Incluye letras acentuadas o símbolos internacionales	`'â'`, `'ñ'`, `'Ω'`	`'ab'`
8️⃣ No se puede dejar vacío	Debe haber exactamente **un carácter** entre comillas	`'a'`	`''` ❌

EJEMPLO RESUELTO

Nº	Literal	¿Válido?	Explicación
1	'a'	☑	Carácter simple válido.
2	'$'	☑	Carácter especial válido.

Nº	Literal	¿Válido?	Explicación
3	'\n'	☑	Secuencia de escape válida (salto de línea).
4	'/n'	✗	No es escape válido, se interpreta como dos caracteres.
5	'\'	☑	Secuencia de escape válida para la barra invertida.
6	'\ñ'	✗	No existe secuencia de escape con `ñ`.
7	"T"	✗	Comillas dobles → literal `String`, no `char`.
8	'ñ'	☑	Carácter Unicode válido de un solo símbolo.
9	'xyz'	✗	Demasiados caracteres (solo se permite uno).
10	'\u0066'	☑	Unicode válido, representa el carácter `'f'`.
11	"XYZ"	✗	Comillas dobles → literal `String`.
12	'4'	☑	Carácter numérico válido.
13	'\t'	☑	Tabulación horizontal, escape válido.
14	'\b'	☑	Retroceso (backspace), escape válido.
15	k	✗	Falta comillas simples → no es literal `char`.
16	+	✗	Falta comillas simples → operador, no `char`.
17	'+'	☑	Carácter `+` válido dentro de comillas simples.
18	'?'	☑	Carácter válido.
19	'â'	☑	Carácter Unicode válido.
20	':'	☑	Carácter válido.

Ejercicio 3 - Declaraciones de variables

A. Escribe la declaración para cada una de las variables y asígnales un valor inicial en la propia declaración de variable

```
// Variables enteras: p, q
int p = 20
int q = 15;

//Variables float: x, y, z
float x = 20f;
float y = 15.00f;
float z = 200f;

//Variables carácter a, b, c
char a = 'H';
char b = 'H';
char c = 'C';
```

```java
//Variable double: raiz1, raiz2
double raiz1 = 2.5;
double raiz2 = -2.5;

// Variable entera larga: contador
long contador = 100000L;

// Variable entera corta: indicador
short indicador = 1200;

// Variable tipo byte: valor
byte valor = 100;

//Variables lógicas: primero, ultimo
boolean primero = true;
boolean ultimo = false;

//Variable tipo cadena de caracteres: nombre
String nombre = "Jose Angel";
```

B. Escribe la declaración más apropiada para cada una de las siguientes variables. El nombre de cada una indica el tipo de dato que contendrá y servirá para determinar el tipo de dato más adecuado en la declaración de la variable

```java
// Variable edad
int edad = 20;

// Variable codigoPostal
String codigoPostal = "08002";

// Variable altura
float altura = 2.5f;

// Variable genero (valores H=hombre , variable mujer = M)
String generoH = "Hombre";
String generoM = "Mujer";

// Variable nombre
String nombre = "Jose Angel";

// Variable númeroDeHijos;
int númeroDeHijos = 4;

// Variable IVA (valores: % a aplicar)
byte IVA = 21;

// Variable peso:
float peso = 60.5f;

// Variable precio:
```

```
float precio = 15.3f;

// Variable alumnoRepetidor (valores: SI/NO)
boolean alumnoRepetidor = true;  // true = si.
boolean alumnoRepetidor = false; // false = no

// Variable mensaje
String mensaje = "Hola, ¿hoy podrás salir?";

// Variable letra
char letra = 'A';

// Variable contador
int contador = 0;

// Variable tallaCamisa (Valores numéricos)
short tallaCamisa = 26;

// Variable tallaCamiseta (valores: S, L, XL...)
char tallaCamiseta = 'S';
char tallaCamiseta = 'L';
String tallaCamiseta = "XL";
```

Ejercicio 4 - Tipodato en expresiones aritméticas y Conversiones

A. A partir de las siguientes declaraciones de variables

```
byte b;
short s;
long ln;
int i, j;
float f;
double dM
char c;
```

Determina cuál es el tipo de dato del resultado de las siguientes expresiones aritméticas

- **Cómo saber el tipo de dato resultando entre dos tipos distintos**:
- Si hay un `double`**, todo se convierte a double**
- Si no hay `double`, pero **sí** `float`**, todo se convierte a** `float`
- Si no hay `float`, pero **si** `long`**, todo se convierte a long**
- Si **solo hay enteros (**`byte`**,** `short`**,** `char`**,** `int`**), todo se convierte a** `int` **automáticamente.**

```
i + c;          //  entero + char = int
f-c;            //  Será float
5.2 /j;         //  double / entero dará un número double.
d + i;          //  double + entero dará un double.
j - 4L * 2.5F;  //  entero - long * float . Dará un float, que es el mayor.
```

```
s + j;          //  short + entero dará un entero.
'a' + c;        //  usar "+" trata los "char" como "int" (enteros), por lo que int
+ int = int
i * f * 2.5;    //  entero * float * double dará un double.
d + c;          //  double + char dará un double.
```

B. A partir de las siguientes declaraciones de variables

```
byte b;
short s;
long ln;
int i, j;
float f;
double d;
char c;
```

Indica en cada caso si la instrucción es válida o si se requiere una conversión explícita (casting) para que compile

Cómo saber cuando la instrucción no es válida y se requiere casting:

- Si es tipo **"mayor"** a **"menor"** → (por ejemplo `int` → `short`, `double` → `float`) **necesitas casting**.
- `Char` → `byte` o `short`
- Si **solo hay enteros (`byte`, `short`, `char`, `int`), todo se convierte a `int` automáticamente.**

```
int z = b + s;          // Válida. byte + short dá un short, que se puede guardar
en un int.
float x = i / f;        // Válida. entero / float es un float. Un float se puede
almacenar en un float.
char ch = c + i;        // Inválida. char + entero da entero ; Hacemos casting.
char ch = (char)(c+i);

long l = ln + i;        //  Válida. long + entero es long, se puede guardar en un
long.
double r = d-f;         //  Válida. double - float es double. Se puede almacenar
en double.
float y = j * 3.5F;     //  Válida. float * float es float. No hace falta casting.
short y = s + 100000;   //  Inválido. short + entero da entero. Hacemos casting.
short y = (short)(s + 100000);

byte nb = b * c;        // Inválido. byte * char da un entero, hacemos casting
para almacenar en byte.
byte nb = (byte)(b*c);

float w = f + d;        // No válido. float + double da double. Double → float
requiere casting.
float w = (float)(f + d);
```

```
double res = ln / 2.0;  // Válido. long + double dará un double. Double se puede almacenar en un double.
```

Tabla resumen sobre cuando se quiere CASTING

Tipo inicial	Tipo(s) final(es)	¿Requiere casting?	Ejemplo de casting	Notas
`short`	`byte`, `char`	Sí	`byte b = (byte) s;`	Puede perder datos (truncamiento)
`int`	`short`, `byte`, `char`	Sí	`short s = (short) i;`	Pérdida de precisión posible
`long`	`int`, `short`, `byte`, `char`	Sí	`int i = (int) l;`	Puede truncar valores grandes
`float`	`int`, `long`, `short`, `byte`, `char`	Sí	`int i = (int) f;`	Pierde decimales (trunca)
`double`	`float`, `int`, `long`, `short`, `byte`, `char`	Sí	`float f = (float) d;`	Pierde precisión/decimales
`char`	`byte`, `short`	Sí	`byte b = (byte) c;`	Puede dar valores negativos o perder info

Tabla resumen sobre Conversión entre tipos de datos en Java

DATO ORIGEN	DATO SALIDA	MÉTODO / EJEMPLO
Primitivo	**Primitivo**	**Casting** → `(tipo) variable` 🧩 *Ejemplo:* `int i = (int) 3.14;`
Objeto (String)	**Primitivo**	**parseTipo()** → `Tipo.parseTipo("valor")` 🧩 *Ejemplo:* `int n = Integer.parseInt("42");`
Primitivo	**Objeto (Wrapper)**	**valueOf()** → `Tipo.valueOf(valor)` 🧩 *Ejemplo:* `Integer num = Integer.valueOf(25);`
Objeto (Wrapper)	**Primitivo**	**xxxValue()** → `objeto.xxxValue()` 🧩 *Ejemplo:* `int n = num.intValue();`
Primitivo	**Cadena (String)**	**String.valueOf()** → `String.valueOf(valor)` 🧩 *Ejemplo:* `String s = String.valueOf(123);`
Objeto (Wrapper)	**Cadena (String)**	**toString()** → `objeto.toString()` 🧩 *Ejemplo:* `String s = num.toString();`

💡 **Ejemplos prácticos**

```java
// Primitivo a primitivo (con casting)
double d = 9.8;
int i = (int) d;   // i = 9

// String a primitivo
int edad = Integer.parseInt("25");

// Primitivo a objeto
Float f = Float.valueOf(3.14f);

// Objeto a primitivo
double x = f.doubleValue();

// Objeto a objeto (mismo tipo)
Integer a = Integer.valueOf(10);
Integer b = Integer.valueOf(a);

// Primitivo a String
String texto = String.valueOf(100);

// String a objeto
Double num = Double.valueOf("123.45");
```

Ejercicio 5 - Operadores lógicos

Si X, Y y Z son variables de tipo boolean con valores X = true, Y = false, Z = true, determina el valor de las siguientes expresiones lógicas:

```java
// Operadores lógicos o puertas lógicas: Son AND, OR y NOT.
// AND -> && (Doble ampersand)          | Sintaxis a && b
// OR  -> || (Doble barra)              | Sintaxis a || b
// NOT -> !  (Signo de exclamación)     | Sintaxis !a

// Para las siguientes variables tipo boolean:
boolean x = true;
boolean y = false;
boolean z = true;

// Determina el valor de las siguientes expresiones lógicas:

boolean resultado1= (x && y) || (x && z);       // True → Si (x AND y) OR (x AND
z) son true, entonces true.

boolean resultado2= (x || !y) && (!x || z);     // True → Si (x OR y!) true AND
(x! OR z) son true, entonces true.

boolean resultado3= (x || y && z);              // True → Si X true OR (y AND
z)true, entonces true.

boolean resultado4= !(x || y) && z;             // False → Si NO(x OR y) AND z son
```

```
true, entonces true. Los valores negados de X e Y n

boolean resultado5= (y || x && !z && !y);       // False → Si Y es true OR (X AND
noZ AND noY) son true, entonces true. Por tanto, false.

boolean resultado6= (!x || z && x && !y);       // True → Si noX OR (z AND x AND
noY) son true, entonces true. Por tanto, true.
```

Ejercicio 6 - Operadores relacionales

A partir de las siguientes declaraciones de variables

```
int i = 8;
int j = 5;
float x = 0.005F;
float y = -0.01F;
char c = 'c';
char d = 'd';
```

Determina el valor de las siguientes expresiones

```
i <= j;              // falso.
c > d;               // falso, porque en codigo ASCII, las letras aumentan su
valor en el abecedario, y las minúsculas valen mas que las mayúsculas.
x >= 0;              // Verdadero. Es mayor.
x < y--;             // (primero comparas, luego decrementas) falso, x vale más.
j != y;              // Verdadero, si son diferentes.
c == 99;             // Verdadero, 'c' en ASCII vale 99.
!(i <= j);           // Verdadero, "!" cambia el valor, entonces 5 vale menos que
8.
!(c == 99);          // Verdadero, "c" minúscula en ascii vale 99.
!(x > 0);            // Falso. El valor dado a X es mayor,al invertir símbolos,
deja de cumplirse true.
-j == i-13;          // Verdadero. -5 es igual a 5-13.
++x > 0;             // Verdadero. Primero hace el incremento, y luego hace la
operación con el valor ya incrementado.
y-- < 1;             // Verdadero. Primero hace la comparación, y tras obtener
true o false, se hace el decremento.
c > d || c > 0;      // Verdadero. La primera condición es false, la segunda es
true, así que true.
5 * (i + j) > 'c';   // Falso. El resultado de la operación de la primera
condición no es mayor al valor de 'c en ASCII'.

//Corregido:
i <=j;               //  Correcto.
c > d;               //  Correcto.
x >= 0;              //  Correcto.
x < y--;             //  Correcto.
```

```
j != y;              //  Correcto. Es distinto.
c == 99;             //  Correcto. 'c' en ASCII vale 99.
!(i <= j);           //  Correcto.
!(c == 99);          //  Correcto.
!(x > 0);            //  Correcto. Es false.
-j == i-13;          //  Correcto.
++x > 0;             //  Correcto. ++x (que es 1.005) es mayor a 0.
y-- < 1;             //  Correcto.
c > d || c > 0;      //  Correcto.
5 * (i + j) > 'c';   //  Correcto.
```

Ejercicio 7 - Expresiones algorítmicas

Escribe las expresiones algorítmicas equivalentes para cada uno de los siguientes enunciados

```
// 1. Comprobar si el contenido de la variable precio es igual o mayor que 10€ y
menor que 50€
float precio = 12.3F;
boolean resultado1 = (precio>=10 && precio<50);     // Uso el operador lógico &&
para comprobar que el valor está dentro del rango abierto [10, 50).
System.out.println(resultado1);

// 2. Comprobar si una variable A de tipo carácter contiene una letra mayúscula.
char A = 'y';
boolean resultado2 = (A>=65 && A<=90);
System.out.println(resultado2);  // Dará false, no hay letra mayuscla.

// El valor asignado a la variable A debe estar entre 65 y 90 inclusive, porque
son los códigos ASCII de las letras mayúsculas de la 'A' a la 'Z'.

// 3. Modificar el valor de la variable entera X duplicando su valor.
// Por ejemplo si N contiene el valor de 10, después de la operación contendrá el
valor de 20.
int x = 10;
x = x*2;
System.out.println(x);

// 4. La diferencia entre el producto de A por B y la suma de C más D.
int a = 10;
int b = 15;
int c = 20;
int d = 8;

System.out.println(a*b -(c+d));

// 5. La suma de la última cifra de los números enteros N y M.
// Sería mas correcto hacer:
int N = 20;
int M = 25;
int suma = (N%10) + (M%10);         // Los paréntesis no son necesarios, porque
```

```
siempre se harán primero los "%" antes que la suma, por preferencia.
System.out.println(suma);

// 6. La suma de los dígitos de un número entero P de 3 cifras.
int P = 123;
suma = P/100+(P%100)/10 + P%10;
System.out.println(suma);

// Lo que hemos hecho es dividir sucesivamente para extraer la centena(1), la
decena(2) y la unidad(3) y después sumar los tres valores.
// Y luego sumamos los restos. | P/100 = 1 ; (P%100)/10 es igual a 2 ; P%10 es
igual a 3.

// 7. Quitarle a un número entero J de 5 cifras su primera cifra. Por ejemplo, si
J contiene valor 12345, después de la operación contendrá el valor 2345.
int J = 12345;
J = J%10000;        // El operador "%" nos devuelve el resto de dividir J entre
10000, eliminando la primera cifra de un número de 5 dígitos.
System.out.println(J);
```